Andromeda Spec 02: Fetch and Branch Prediction

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `ifu`, `bpu`, `fetch_buffer` · **Milestone:** Instruction fetch and branch prediction

1. What this unit does

The front end keeps the rest of the core supplied with instructions. Each cycle it fetches one 16-byte block (4 instructions), asks the branch predictor where the program goes next, and writes the useful instructions into a 16-entry fetch buffer that decode reads from.

The fetch unit (`ifu`) and fetch buffer are fully specified here. For branch prediction, we'll just keep assuming PC+4 for the simplicity considering the overall workload, plus the use of Return Address Stack (RAS) which is a simple piece of hardware.

2. How it works

2.1 Fetch blocks

- Fetch addresses are 16-byte aligned, so one request returns 4 instruction slots. When the PC isn't aligned (after a redirect or taken branch), the slots below it are marked invalid. For instance branch target is 0x4, but you can only fetch at 0x0 for 16 byte alignment, 0x0 instruction will be marked invalid.
- Calls and returns are special: a call's return address is pushed onto a return address stack (RAS), and a return predicts its target by popping it. This is far more accurate than remembering a return's last target.

2.2 Fetch flow

1. **F1:** the next fetch PC is chosen, in priority order: redirect from branch instruction execution, predicted target from the `RAS`, or the next sequential block. The PC goes to instruction memory.
2. **F2:** instructions return. The `ifu` applies the PC (invalidating slots outside the block's valid range), attaches the prediction fields to each slot, and writes the block into the fetch buffer.
3. Up to 2 requests may be in flight. Each carries an epoch bit that flips on every redirect, and responses tagged with an old epoch are dropped.

3. Interface

3.1 ifu (fetch unit)

Signal	Dir	Width	Description
`boot_addr`	in	32	PC after reset
`flush`	in	1	`fe_flush` from pipe_ctrl

Signal	Dir	Width	Description
`redirect_valid` / `redirect_pc`	in	1 / 32	Restart fetch at this PC
`imem_req_valid` / `imem_req_ready` / `imem_req_addr`	out / in / out	1 / 1 / 32	Instruction memory request
`imem_resp_valid` / `imem_resp_data` / `imem_resp_err`	in	1 / 128 / 1	Instruction memory response
`ras_push` / `ras_pop` / `ras_push_addr`	out	1 / 1 / 32	Speculative return stack update
`fb_enq_valid`	out	1	Block ready for the fetch buffer
`fb_enq_ready`	in	1	Fetch buffer has at least 4 free entries
`fb_enq_slot`	out	4 × fetch_slot_t	The 4 slots, with per-slot valid

3.2 fetch_buffer

A FIFO that accepts up to 4 slots per cycle, compacting out invalid ones, and hands decode the oldest group of up to 4. Use the given FIFO ip for implementation.

4. Implementation notes

- **Fetch buffer compaction.** Packing up to 4 valid slots out of a block into consecutive entries is a small shifter network. An alternative is to store whole blocks with per-slot valid bits and have the read side skip invalid slots; this moves the complexity to the decode side. Pick one and keep it inside the module.
- **Epoch, not tags.** One epoch bit per in-flight request is enough with at most 2 in flight, since a redirect bumps the epoch and every older response is discarded.
- **Memory alignment.** If the memory side can only return aligned 16-byte blocks, fetch from the middle of a block simply wastes slots. That's acceptable for v1.
- **Faults.** `imem_resp_err` marks every slot from that block with `fetch_err`. Decode turns that into an instruction access fault, raised when the instruction reaches the ROB head, so a fault on a wrong-path fetch is harmless.
- **Keep the predictor swappable.** Write the `bpu` so the direction predictor is a submodule behind a small internal interface. That makes it easy to compare candidates in RTL as well as in the software model.

5. Verification checklist

- ☐ Sequential fetch across many blocks with random memory latency
- ☐ Redirect while 2 requests are in flight: stale responses are dropped
- ☐ Redirect to an unaligned PC: lower slots invalid
- ☐ Fetch buffer full: fetch holds without losing or duplicating slots

- ☐ Fetch fault on a wrong-path block causes no trap
- ☐ Core passes all tests with the always-not-taken stub
- ☐ Chosen predictor’s RTL matches its software model on recorded traces
- ☐ RAS correct across nested calls, overflow and flushes